COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 10, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 SORTING ALGORITHMS PART 1

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble and insertion sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted.
- Create a header file for the implementation of the different sorting algorithms. Use the created array for each sorting algorithm.
- Import this header file into your main.cpp.

Show the output of the preparation task in the table.
(Table 7-1. Array of Values for Sort Algorithm Testing)

Screenshot	**My Code: (main.cpp)** ILOA > main.cpp > ... 1 #include <iostream> 2 #include <cstdlib> 3 #include <ctime> 4 #include "sortingAlgo.h" 5 6 const int maxSize = 100; 7 8 void displayArr(int arr[]); 9 10 int main() { 11 12 // Create an array with 100 unsorted elements 13 int unsortedArr[maxSize]; 14 15 // Use time(0) as seed for random values 16 srand(time(0));

```
17
18      // Populate the array with random values
19      for (int i = 0; i < maxSize; i++) {
20          unsortedArr[i] = rand() % 100;
21      }
22
23      // Display the original array
24      std::cout << "\nArray:" << std::endl;
25      std::cout << "-----------------------------------" << std::endl;
26      displayArr(unsortedArr);
27      std::cout << std::endl;
28  }
29
30  // Display all elements in the array
31  void displayArr(int arr[]) {
32      for (int i = 0; i < maxSize; i++) {
33          std::cout << arr[i] << " ";
34      }
35      std::cout << std::endl;
36  }
```

Output

```
Array:
-----------------------------------
72 95 3 52 16 70 70 73 20 0 9 45 31 87 33 37 1
 11 13 87 50 6 62 41 21 49 74 81 41 75 0 78 45
 9 21 27 97 35 33 14 64 83 57 7 33 91 59 30 43
 71 19 34 12 39 93 68 63 98 67 84 97 28 88 33
17 21 1 31 3 14 71 60 63 27 9 24 13 46 83 78 8
 44 79 98 19 17 76 68 68 21 72 45 27 0 14 53 2
9 65 87 56
```

Observations

The code successfully created 100 random values using ***rand() % 100***, then stored the values in ***unsortedArr[maxSize]***. The ***for*** loop was used to fill all 100 positions, while ***displayArr(unsortedArr)*** showed the values in their original unsorted order. Some numbers can appear more than once because random values from 0 to 99 are allowed. Overall, the output shows that the array was prepared properly and can now be used for the different sorting algorithms.

Table 7-1. Array of Values for Sort Algorithm Testing

A.1. Bubble Sort Technique

Using the bubble sort technique, sorting is done in passes or iteration. Thus at the end of each iteration, the heaviest element is placed at its proper place in the list. In other words, the largest element in the list bubbles up.

Include a screenshot of your code and output console along with your observations in the table.
(Table 7-2. Bubble Sort Technique)

Code	**My Code: (sortingAlgo.h)**
	ILOA > C sortingAlgo.h > routine_smallest<T>(T [], int, int)
	`#ifndef SORTINGALGO_H`
	`#define SORTINGALGO_H`
	`#include <iostream>`
	
	`// Sort the array using Bubble Sort`
	`template <typename T>`
	`void bubbleSort(T arr[], int maxSize) {`
	
	`    // Compare each element with the elements after it`
	`    for (int i = 0; i < maxSize; i++) {`
	`        for (int j = i + 1; j < maxSize; j++) {`
	
	`            // Swap the elements if the current element is greater`
	`            if (arr[j] < arr[i]) {`
	`                std::swap(arr[i], arr[j]);`
	`            }`
	`        }`
	`    }`
	`}`
	
	My Code: (main.cpp)
	`    // Sort the array using Bubble Sort`
	`    std::cout << "\nBubble Sort:" << std::endl;`
	`    std::cout << "---------------------------------" << std::endl;`
	`    bubbleSort(bubbleArr, maxSize);`
	`    displayArr(bubbleArr);`

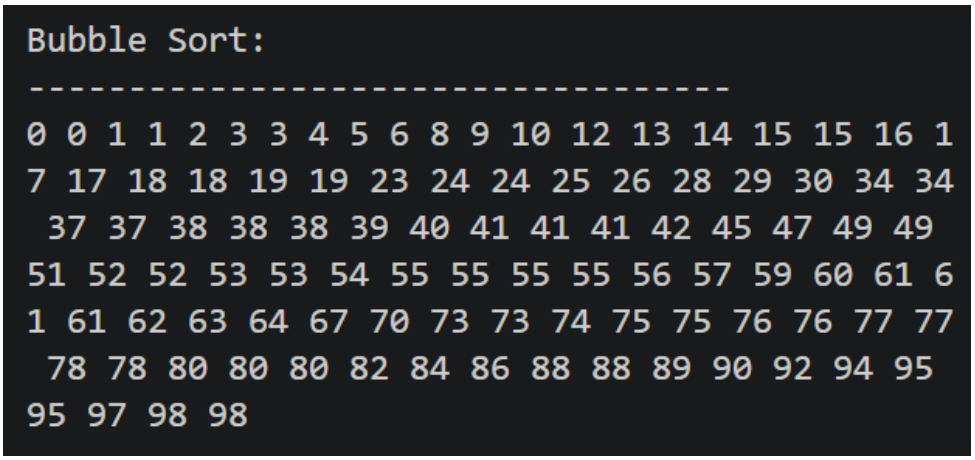
Output	
Observations	The code successfully sorted the 100 random values using the ***bubbleSort(bubbleArr, maxSize)*** function, where the nested ***for*** loops compare the values and ***std::swap(arr[i], arr[jj])*** changes their positions when a smaller value is found. The ***bubbleArr*** was first copied from ***unsortedArr***, so the original random values stayed unchanged for comparison with the other sorting methods. After running ***bubbleSort()***, the output showed the values arranged from smallest to largest, which shows that the sorting process worked properly. The process can be seen as repeated comparisons through the array, where smaller values move toward the proper position during each pass.

Table 7-2. Bubble Sort Technique

A.2. Selection Sort Algorithm

Selection sort is quite a straightforward sorting technique as the technique only involves finding the smallest element in every pass and placing it in the correct position. Selection sort works efficiently when the list to be sorted is of small size but its performance is affected badly as the list to be sorted grows in size. Hence we can say that selection sort is not advisable for larger lists of data.

Include a screenshot of your code and output console along with your observations in the table.
(Table 7-3. Selection Sort Algorithm)

Screenshot	**My Code: (sortingAlgo.h)**
	
	21 // Find the position of the smallest element
	22 template <typename T>
	23 int routine_smallest(T arr[], int k, int arrSize){
	24
	25 // Start the smallest position at k
	26 int position = k;
	27
	28 // Store the current smallest element
	29 T smallestElement = arr[k];
	30
	31 // Check the remaining elements in the array
	32 for(int j = k+1; j < arrSize; j++){
	33
	34 // Update the smallest element and its position
	35 if(arr[j] < smallestElement){
	36 smallestElement = arr[j];
	37 position = j;
	38 }
	39 }
	40
	
	41 // Return the position of the smallest element
	42 return position;
	43 }
	44
	45 // Sort the array using selection sort
	46 template <typename T>
	47 void selectionSort(T arr[], int arrSize){
	48 int min, temp;

```
49
50      // Find the smallest element for each position
51      for(int i = 0; i < arrSize; i++ ){
52
53          // Get the position of the smallest element
54          min = routine_smallest(arr, i, arrSize);
55
56          // Swap the current element with the smallest element
57          temp = arr[i];
58          arr[i] = arr[min];
59          arr[min] = temp;
60      }
61  }
```

My Code: (main.cpp)

```
44      // Sort the array using Selection Sort
45      std::cout << "\nSelection Sort:" << std::endl;
46      std::cout << "----------------------------------" << std::endl;
47      selectionSort(selectionArr, maxSize);
48      displayArr(selectionArr);
49
```

Output

```
Selection Sort:
----------------------------------
0 1 1 1 1 2 2 3 4 5 6 6 7 8 8 11 11 12 14 15 1
5 16 17 19 19 21 22 26 28 28 31 31 32 33 33 36
 36 37 37 38 39 39 41 42 42 42 42 44 45 47 48
49 49 50 51 52 53 53 53 54 54 54 56 60 60 62 6
2 64 65 65 69 69 69 71 71 72 74 75 76 76 77 79
 79 79 80 80 81 82 84 85 89 89 91 92 92 94 95
98 98 98
```

Observations

The code successfully sorted the 100 random values using the ***selectionSort(selection Arr, maxSize)*** function. The routine_smallest(arr, i, arrSize) function was important because the function checks the remaining values and finds the position of the smallest element for every

	pass. After finding the smallest value, the code uses temp to swap the value at arr[i] with the value at arr[min], so the smallest value is placed in the correct position. The output showed the values arranged from smallest to largest, and the process was repeated until all 100 values were sorted properly. This also shows how Selection Sort searches for the smallest value first before placing the value in the proper position.

Table 7-3. Selection Sort Algorithm

A.3. Insertion Sort Algorithm

In the insertion sort technique, we start from the second element and compare it with the first element and put it in a proper place. Then we perform this process for the subsequent elements. We compare each element with all its previous elements and put or insert the element in its proper position. Insertion sort technique is more feasible for arrays with a smaller number of elements. It is also useful for sorting linked lists. Linked lists have a pointer to the next element (in case of a singly linked list) and a pointer to the previous element as well (in case of a doubly linked list). Hence it becomes easier to implement insertion sort for a linked list.

Include a screenshot of your code and output console along with your observations in the table 7-4.
(Table 7-4. Insertion Sort Algorithm)

Screenshot	**My Code: (sortingAlgo.h)**
	
	63 // Sort the array using insertion sort
	64 template <typename T>
	65 void insertionSort(T arr[], int arrSize){
	66 int k = 1, j;
	67 T temp;
	68
	69 // Start from the second element
	70 while(k < arrSize){
	71
	72 // Store the current element
	73 temp = arr[k];
	74 j = k - 1;
	75
	76 // Move larger elements one position to the right
	77 while(j >= 0 && temp <= arr[j]){
	78 arr[j + 1] = arr[j];

```
79                  j--;
80              }
81
82              // Place the current element in the correct position
83              arr[j + 1] = temp;
84              k++;
85          }
86      }
87
88      #endif
```

My Code: (main.cpp)

```
50      // Sort the array using Insertion Sort
51      std::cout << "\nInsertion Sort:" << std::endl;
52      std::cout << "----------------------------------" << std::endl;
53      insertionSort(insertionArr, maxSize);
54      displayArr(insertionArr);
55
56      std::cout << std::endl;
57  }
58
```

Output

```
Insertion Sort:
----------------------------------
0 1 4 5 6 6 8 8 9 11 15 15 20 20 22 22 22 23 2
3 25 25 26 28 28 29 29 30 31 32 33 34 35 37 40
 41 42 44 45 45 46 46 46 46 46 48 53 55 55 57
59 60 60 61 61 61 62 62 66 66 66 66 68 69 71 7
2 74 74 76 77 77 78 78 79 79 80 81 81 82 84 86
 86 86 86 86 86 87 87 87 90 90 90 91 92 93 94
94 96 96 98 99
```

Observations

The code successfully sorted the 100 random values using the ***insertionSort(insertion Arr, maxSize)*** function. The process starts from the second element because ***int k = 1*** is used, then ***temp = arr[k]*** stores the current value before comparing with the previous elements. The

	while(j >= 0 && temp <= arr[j]) condition is important because larger values are moved one position to the right until the proper position is found. After that, ***arr[j + 1] = temp*** places the current value in the correct position. The output showed the values arranged from smallest to largest, showing that the insertion process worked properly even with the 100 random values.

Table 7-4. Insertion Sort Algorithm

My Output: (Complete)

```
Array:
---------------------------------
30 58 40 90 18 69 56 63 19 0 42 91 84 80 10 27 37 65 88 9 73 77 87 3 91 20
97 71 62 84 55 80 49 16 35 88 59 80 68 42 10 26 86 10 83 16 99 55 45 1 54 4
1 67 21 89 67 12 57 80 38 67 2 36 26 96 56 49 21 15 6 3 34 47 65 26 71 28 5
8 69 94 49 75 21 24 45 72 23 83 46 66 51 66 98 13 45 99 66 19 60 12

Bubble Sort:
---------------------------------
0 1 2 3 3 6 9 10 10 10 12 12 13 15 16 16 18 19 19 20 21 21 21 23 24 26 26 2
6 27 28 30 34 35 36 37 38 40 41 42 42 45 45 45 46 47 49 49 49 51 54 55 55 5
6 56 57 58 58 59 60 62 63 65 65 66 66 66 67 67 67 68 69 69 71 71 72 73 75 7
7 80 80 80 80 83 83 84 84 86 87 88 88 89 90 91 91 94 96 97 98 99 99

Selection Sort:
---------------------------------
0 1 2 3 3 6 9 10 10 10 12 12 13 15 16 16 18 19 19 20 21 21 21 23 24 26 26 2
6 27 28 30 34 35 36 37 38 40 41 42 42 45 45 45 46 47 49 49 49 51 54 55 55 5
6 56 57 58 58 59 60 62 63 65 65 66 66 66 67 67 67 68 69 69 71 71 72 73 75 7
7 80 80 80 80 83 83 84 84 86 87 88 88 89 90 91 91 94 96 97 98 99 99

Insertion Sort:
---------------------------------
0 1 2 3 3 6 9 10 10 10 12 12 13 15 16 16 18 19 19 20 21 21 21 23 24 26 26 2
6 27 28 30 34 35 36 37 38 40 41 42 42 45 45 45 46 47 49 49 49 51 54 55 55 5
6 56 57 58 58 59 60 62 63 65 65 66 66 66 67 67 67 68 69 69 71 71 72 73 75 7
7 80 80 80 80 83 83 84 84 86 87 88 88 89 90 91 91 94 96 97 98 99 99
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

Note: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

My Work:	
Pseudocode	**votingSort.h** FUNCTION displayArr(arr, maxSize) FOR i = 0 TO maxSize - 1 DO PRINT arr[i] END FOR PRINT new line END FUNCTION

```
FUNCTION bubbleSort(arr, maxSize)
   FOR i = 0 TO maxSize - 1 DO
      FOR j = i + 1 TO maxSize - 1 DO
         IF arr[j] is less than arr[i] THEN
            SWAP arr[i] and arr[j]
         END IF
      END FOR
   END FOR
END FUNCTION


FUNCTION countVotes(arr, maxSize, candidate)
   SET count = 0

   FOR i = 0 TO maxSize - 1 DO
      IF arr[i] is equal to candidate THEN
         INCREASE count by 1
      END IF
   END FOR

   RETURN count
END FUNCTION
```

main_ILOB.cpp

```
START

SET maxSize = 100
DECLARE unsortedArr[maxSize]

INITIALIZE random number generator using current time

FOR i = 0 TO maxSize - 1 DO
   ASSIGN unsortedArr[i] a random number from 1 to 5
END FOR

PRINT "Original List of Votes"
PRINT separator line
CALL displayArr(unsortedArr, maxSize)
```

```
PRINT "Sorted List of Votes"
PRINT separator line
CALL bubbleSort(unsortedArr, maxSize)
CALL displayArr(unsortedArr, maxSize)

COMPUTE vote1 = countVotes(unsortedArr, maxSize, 1)
COMPUTE vote2 = countVotes(unsortedArr, maxSize, 2)
COMPUTE vote3 = countVotes(unsortedArr, maxSize, 3)
COMPUTE vote4 = countVotes(unsortedArr, maxSize, 4)
COMPUTE vote5 = countVotes(unsortedArr, maxSize, 5)

PRINT "Vote Count"
PRINT separator line
PRINT "Candidate 1: ", vote1
PRINT "Candidate 2: ", vote2
PRINT "Candidate 3: ", vote3
PRINT "Candidate 4: ", vote4
PRINT "Candidate 5: ", vote5

SET highest = vote1
SET winner = 1

IF vote2 is greater than highest THEN
   SET highest = vote2
   SET winner = 2
END IF

IF vote3 is greater than highest THEN
   SET highest = vote3
   SET winner = 3
END IF

IF vote4 is greater than highest THEN
   SET highest = vote4
   SET winner = 4
END IF

IF vote5 is greater than highest THEN
   SET highest = vote5
```

	SET winner = 5 END IF PRINT "Winning Candidate" PRINT separator line IF winner is equal to 1 THEN PRINT "Candidate 1 - Bo Dalton Capistrano" ELSE IF winner is equal to 2 THEN PRINT "Candidate 2 - Cornelius Raymon Agustín" ELSE IF winner is equal to 3 THEN PRINT "Candidate 3 - Deja Jayla Bañaga" ELSE IF winner is equal to 4 THEN PRINT "Candidate 4 - Lalla Brielle Yabut" ELSE IF winner is equal to 5 THEN PRINT "Candidate 5 - Franklin Relano Castro" END IF PRINT "Number of Votes: ", highest END

Screenshot of Code	**My Code: (votingSort.h)**
	
	ILOB > votingSort.h > ...
	1 #ifndef VOTINGSORT_H
	2 #define VOTINGSORT_H
	3
	4 #include <iostream>
	5
	6 // Display all elements in the array
	7 template <typename T>
	8 void displayArr(T arr[], int maxSize) {
	9
	10 // Display each element in the array
	11 for (int i = 0; i < maxSize; i++) {
	12 std::cout << arr[i] << " ";
	13 }
	14
	15 std::cout << std::endl;
	16 }

```
17
18  // Sort the array using Bubble Sort
19  template <typename T>
20  void bubbleSort(T arr[], int maxSize) {
21
22      // Compare each element with the elements after it
23      for (int i = 0; i < maxSize; i++) {
24          for (int j = i + 1; j < maxSize; j++) {
25
26              // Swap the elements if the current element is greater
27              if (arr[j] < arr[i]) {
28                  std::swap(arr[i], arr[j]);
29              }
30          }
31      }
32  }
33
34  // Count the votes for a candidate
35  template <typename T>
36  int countVotes(T arr[], int maxSize, int candidate) {
37      int count = 0;
38
```

```
39      // Check each element in the array
40      for (int i = 0; i < maxSize; i++) {
41
42          // Count the vote if it matches the candidate number
43          if (arr[i] == candidate) {
44              count++;
45          }
46      }
47
48      // Return the total number of votes
49      return count;
50  }
51
52  #endif
```

My Code: (main_ILOB.cpp)

ILOB > main_ILOB.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "votingSort.h"

const int maxSize = 100;

int main() {

    // Create an array with 100 unsorted votes
    int unsortedArr[maxSize];

    // Use time(0) as seed for random values
    srand(time(0));

    // Populate the array with random votes from 1 to 5
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % 5 + 1;
    }

    // Display the original list of votes
    std::cout << "\nOriginal List of Votes:" << std::endl;
    std::cout << "---------------------------------------" << std::endl;
    displayArr(unsortedArr, maxSize);

    // Sort the votes using Bubble Sort
    std::cout << "\nSorted List of Votes:" << std::endl;
    std::cout << "---------------------------------------" << std::endl;
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr, maxSize);
```

```cpp

    // Count the votes for each candidate
    int vote1 = countVotes(unsortedArr, maxSize, 1);
    int vote2 = countVotes(unsortedArr, maxSize, 2);
    int vote3 = countVotes(unsortedArr, maxSize, 3);
    int vote4 = countVotes(unsortedArr, maxSize, 4);
    int vote5 = countVotes(unsortedArr, maxSize, 5);

    // Display the vote count for each candidate
    std::cout << "\nVote Count:" << std::endl;
    std::cout << "---------------------------------------" << std::endl;
    std::cout << "Candidate 1: " << vote1 << std::endl;
    std::cout << "Candidate 2: " << vote2 << std::endl;
    std::cout << "Candidate 3: " << vote3 << std::endl;
    std::cout << "Candidate 4: " << vote4 << std::endl;
    std::cout << "Candidate 5: " << vote5 << std::endl;

```

```cpp
    // Set the first candidate as the current highest
    int highest = vote1;
    int winner = 1;

    // Check if Candidate 2 has more votes
    if (vote2 > highest) {
        highest = vote2;
        winner = 2;
    }

    // Check if Candidate 3 has more votes
    if (vote3 > highest) {
        highest = vote3;
        winner = 3;
    }

    // Check if Candidate 4 has more votes
    if (vote4 > highest) {
        highest = vote4;
        winner = 4;
    }
```

```cpp
    // Check if Candidate 5 has more votes
    if (vote5 > highest) {
        highest = vote5;
        winner = 5;
    }

    // Display the winning candidate
    std::cout << "\nWinning Candidate:" << std::endl;
    std::cout << "----------------------------------------" << std::endl;

    if (winner == 1) {
        std::cout << "Candidate 1 - Bo Dalton Capistrano" << std::endl;
    }
    else if (winner == 2) {
        std::cout << "Candidate 2 - Cornelius Raymon Agustín" << std::endl;
    }
    else if (winner == 3) {
        std::cout << "Candidate 3 - Deja Jayla Bañaga" << std::endl;
    }
    else if (winner == 4) {
        std::cout << "Candidate 4 - Lalla Brielle Yabut" << std::endl;
    }
    else if (winner == 5) {
        std::cout << "Candidate 5 - Franklin Relano Castro" << std::endl;
    }
```

```
95      std::cout << "Number of Votes: " << highest << std::endl << std::endl;
96
97      return 0;
98  }
```

Output

My Output 1:

```
Original List of Votes:
----------------------------------------
5 4 3 1 1 4 3 2 5 4 2 3 2 3 4 4 2 1 4 1 1 1 5 5 2 4 1 2 2 4 5 1 2 4 5
 5 4 2 1 3 4 2 1 3 3 5 3 3 4 2 2 3 3 3 5 3 4 5 4 2 2 2 1 1 1 4 3 3 4
1 2 4 1 5 1 3 2 4 4 3 4 5 2 1 1 5 1 1 4 1 1 1 1 3 4 3 3 1 4 5

Sorted List of Votes:
----------------------------------------
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Count:
----------------------------------------
Candidate 1: 25
Candidate 2: 18
Candidate 3: 20
Candidate 4: 23
Candidate 5: 14

Winning Candidate:
----------------------------------------
Candidate 1 - Bo Dalton Capistrano
Number of Votes: 25
```

My Output 2:

```
Original List of Votes:
----------------------------------------
1 3 4 2 2 1 1 2 3 4 2 2 4 4 2 3 1 2 5 3 4 3 1 4 3 1 5 4 2 5 5 1 1 5 2
 5 3 1 3 1 4 4 2 4 4 3 2 2 3 5 5 4 1 3 4 3 4 3 5 1 2 1 1 4 1 5 4 3 4
5 4 3 5 3 2 4 4 3 3 5 1 5 4 3 5 1 2 1 3 2 1 5 4 3 1 3 1 1 5 3
```

```
Sorted List of Votes:
---------------------------------------
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Count:
---------------------------------------
Candidate 1: 22
Candidate 2: 16
Candidate 3: 23
Candidate 4: 22
Candidate 5: 17

Winning Candidate:
---------------------------------------
Candidate 3 - Deja Jayla Bañaga
Number of Votes: 23
```

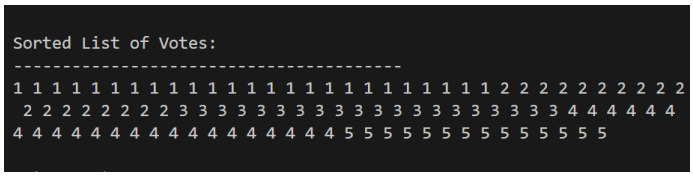
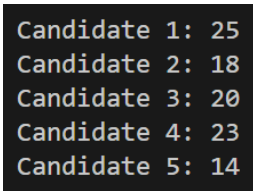
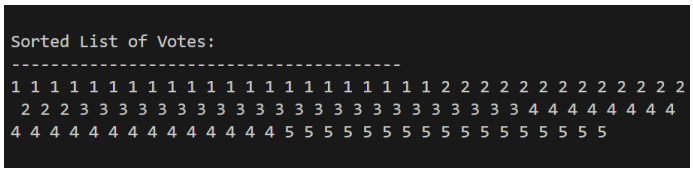
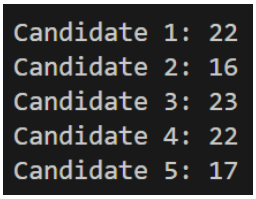
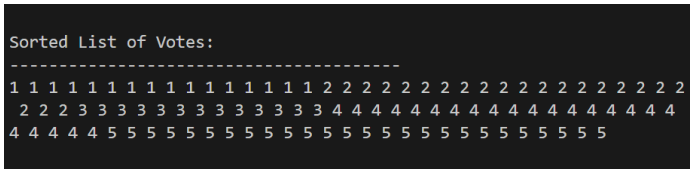
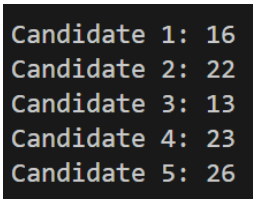
My Output 3:

```
Original List of Votes:
---------------------------------------
5 4 4 2 2 5 4 1 5 4 4 2 3 1 5 5 2 1 2 2 1 3 3 1 3 2 5 5 2 2 3 4 5 4 4
 4 3 4 4 3 5 5 5 2 4 1 2 5 1 3 5 2 4 5 5 1 1 1 3 2 4 4 2 4 3 5 5 5 1
2 1 5 2 5 4 2 4 4 1 3 5 3 5 5 4 2 2 4 3 5 1 5 4 4 1 5 2 2 2 1

Sorted List of Votes:
---------------------------------------
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Vote Count:
---------------------------------------
Candidate 1: 16
Candidate 2: 22
Candidate 3: 13
Candidate 4: 23
Candidate 5: 26
```

	``` Winning Candidate: -- Candidate 5 - Franklin Relano Castro Number of Votes: 26 ```
Observation	The code successfully generated 100 random vote values from 1 to 5 using ***rand() % 5 + 1***, which represents the five candidates. The ***displayArr()*** function was used to show the original and sorted vote list, while the ***bubbleSort()*** function arranged the values from smallest to largest using the sorting process. After sorting, the ***countVotes()*** function checked the whole array and counted how many votes matched each candidate number from 1 to 5. The ***highest*** and ***winner*** variables were then used in the ***main()*** function to compare all five vote counts and identify the candidate with the highest number of votes. The final ***if*** and ***else if*** statements displayed the matching candidate name and the number of votes. Finally, the code showed that sorting and searching were both used properly to count the votes and determine the winning candidate.
Justification	***Bubble Sort*** was chosen because the code is simple and easy to follow, especially for a small voting problem with only ***100 values***. The sorting process uses repeated comparisons and swaps, making the steps easier to understand and apply in the code. The ***bubbleSort()*** function can arrange the vote values from smallest to largest before the ***countVotes()*** function checks the values for each candidate. Even though ***Bubble Sort*** can take more time when handling a very large amount of data, ***100 vote values*** is still manageable for this activity, so the sorting process can be done properly without making the code too complicated.

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	Candidate 1: 25 Candidate 2: 18 Candidate 3: 20 Candidate 4: 23 Candidate 5: 14	
	Candidate 1: 22 Candidate 2: 16 Candidate 3: 23 Candidate 4: 22 Candidate 5: 17	
	Candidate 1: 16 Candidate 2: 22 Candidate 3: 13 Candidate 4: 23 Candidate 5: 26	

Question: Was your developed vote counting algorithm effective? Why or why not?

The developed vote counting algorithm was effective because the code was able to generate 100 votes, sort the values, count the votes for all five candidates, and show the candidate with the highest vote count. The bubbleSort() function arranged the vote values properly, while the countVotes() function checked each value and counted the matching candidate number. The highest and winner variables then compared the five vote counts and displayed the correct candidate based on the highest count. The process worked properly from the original list until the final result, so the needed vote counting steps were completed even if the code can still be improved for bigger amounts of data.

Conclusion:

The activity helped in learning how sorting techniques work and how they can be applied using C++. For the lessons learned, Bubble Sort compares and swaps values, Selection Sort finds the smallest value and places it in the correct position, while Insertion Sort takes each value and inserts it into the proper place. The activity also showed how the same sorting idea can be used with an array of 100 random values and how different functions can handle each sorting process. For the procedure, the codes were created and tested step by step, starting with generating and storing random values, then applying Bubble Sort, Selection Sort, and Insertion Sort. The important parts such as bubbleSort(), selectionSort(), routine_smallest(), and insertionSort() helped show how each sorting method works. For the supplementary activity, the sorting process was applied to a vote counting problem, where displayArr(), bubbleSort(), and countVotes() were used to arrange the votes, count each candidate's votes, and identify the candidate with the highest vote count.

One difficulty faced was understanding the different ways each sorting technique moves the values, especially when checking the loops and the swapping process. Another difficulty was making sure the pseudocode, code, manual count, and output were all matching, especially in the vote counting activity. Even with these difficulties, the results were correct and the output showed that the sorting and vote counting processes worked properly. Overall, the activity was done well because the sorting techniques were successfully implemented and applied to a simple data problem, but there is still an area for improvement in understanding the differences between each sorting method and writing the code with less confusion. More practice with arrays, loops, swapping, and different sorting problems can help improve the understanding and confidence in using these algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.